Create ArrayList from array

I have an array that is initialized like:

```
Element[] array = {new Element(1), new Element(2), new Element(3)};
```

I would like to convert this array into an object of the ArrayList class.

```
ArrayList<Element> arraylist = ???;
```

java arrays arraylist

edited Aug 28 '15 at 15:52

Francesco Menzani
2,437 6 13 38

asked Oct 1 '08 at 14:38

Ron Tuffin
12.9k 15 47 67

17 Answers

```
new ArrayList<Element>(Arrays.asList(array))
```

answered Oct 1 '08 at 14:39

Tom
26k 2 15 32

206 Yep. And in the (most common) case where you just want a list, the `new ArrayList` call is unecessary as well. – Calum Oct 1 '08 at 14:41

63 @Luron - just use `List<ClassName> list = Arrays.asList(array)` – Pool Jun 29 '11 at 15:18

137 @Calum and @Pool - as noted below in Alex Miller's answer, using `Arrays.asList(array)` without passing it into a new `ArrayList` object will fix the size of the list. One of the more common reasons to use an `ArrayList` is to be able to dynamically change its size, and your suggestion would prevent this. – Code Jockey Sep 26 '11 at 19:04

66 Arrays.asList() is a horrible function, and you should never just use its return value as is. It breaks the List template, so always use it in the form indicated here, even if it does seem redundant. Good answer. – Adam Jan 23 '13 at 3:28

34 @Adam Please study the javadoc for java.util.List. The contract for add allows them to throw an UnsupportedOperationException. docs.oracle.com/javase/7/docs/api/java/util/… Admittedly, from an object-oriented perspective it is not very nice that many times you have to know the concrete implementation in order to use a collection - this was a pragmatic design choice in order to keep the framework simple. – lbalazscs Feb 22 '13 at 9:41

Given:

```
Element[] array = new Element[] { new Element(1), new Element(2), new Element(3) };
```

The simplest answer is to do:

```
List<Element> list = Arrays.asList(array);
```

This will work fine. But some caveats:

1. The list returned from asList has **fixed size**. So, if you want to be able to add or remove elements from the returned list in your code, you'll need to wrap it in a new `ArrayList`. Otherwise you'll get an `UnsupportedOperationException`.
2. The list returned from `asList()` is backed by the original array. If you modify the original array, the list will be modified as well. This may be surprising.

edited Nov 7 '14 at 14:22 — George Garchagudashvili — 2,709 7 17 34

answered Oct 1 '08 at 15:39 — Alex Miller — 37.1k 16 77 125

2 What is the time complexity of both operations? I mean with and without using explicit arraylist construction. – damned May 23 '12 at 2:17

12 Arrays.asList() merely creates an ArrayList by wrapping the existing array so it is O(1). – Alex Miller May 23 '12 at 13:15

7 Wrapping in a new ArrayList() will cause all elements of the fixed size list to be iterated and added to the new ArrayList so is O(n). – Alex Miller May 23 '12 at 13:21

5 the implementation of List returned by asList() does not implement several of List's methods (like add(), remove(), clear(), etc...) which explains the UnsupportedOperationException. definitely a caveat... – sethro Nov 14 '12 at 19:33

(old thread, but just 2 cents as none mention Guava or other libs and some other details)

If You Can, Use Guava

It's worth pointing out the Guava way, which greatly simplifies these shenanigans:

Usage

For an Immutable List

Use the `ImmutableList` class and its `of()` and `copyOf()` factory methods (elements can't be null):

```
List<String> il = ImmutableList.of("string", "elements");  // from varargs
List<String> il = ImmutableList.copyOf(aStringArray);      // from array
```

For A Mutable List

Use the `Lists` class and its `newArrayList()` factory methods:

```
List<String> l1 = Lists.newArrayList(anotherListOrCollection);    // from collection
List<String> l2 = Lists.newArrayList(aStringArray);               // from array
List<String> l3 = Lists.newArrayList("or", "string", "elements"); // from varargs
```

Please also note the similar methods for other data structures in other classes, for instance in `Sets`.

Why Guava?

The main attraction could be to reduce the clutter due to generics for type-safety, as the use of the Guava factory methods allow the types to be inferred most of the time. However, this argument holds less water since Java 7 arrived with the new diamond operator.

But it's not the only reason (and Java 7 isn't everywhere yet): the shorthand syntax is also very handy, and the methods initializers, as seen above, allow to write more expressive code. You do in one Guava call what takes 2 with the current Java Collections.

If You Can't...

For an Immutable List

Use the JDK's `Arrays` class and its `asList()` factory method, wrapped with a `Collections.unmodifiableList()`:

```
List<String> l1 = Collections.unmodifiableList(Arrays.asList(anArrayOfElements));
List<String> l2 = Collections.unmodifiableList(Arrays.asList("element1", "element2"));
```

Note that the returned type for `asList()` is a `List` using a concrete `ArrayList` implementation, but **it is NOT** `java.util.ArrayList`. It's an inner type, which emulates an `ArrayList` but actually directly references the passed array and makes it "write through" (modifications are reflected in the array).

It forbids modifications through some of the `List` API's methods by way of simply extending an `AbstractList` (so, adding or removing elements is unsupported), however it allows calls to `set()` to override elements. Thus this list isn't truly immutable and a call to `asList()` should be wrapped with `Collections.unmodifiableList()`.

See the next step if you need a mutable list.

For a Mutable List

Same as above, but wrapped with an actual `java.util.ArrayList`:

```
List<String> l1  = new ArrayList<String>(Arrays.asList(array));    // Java 1.5 to 1.6
List<String> l1b = new ArrayList<>(Arrays.asList(array));          // Java 1.7+
List<String> l2  = new ArrayList<String>(Arrays.asList("a", "b")); // Java 1.5 to 1.6
List<String> l2b = new ArrayList<>(Arrays.asList("a", "b"));       // Java 1.7+
```

For Educational Purposes: The Good ol' Manual Way

```
// for Java 1.5+
static <T> List<T> arrayToList(final T[] array) {
  final List<T> l = new ArrayList<T>(array.length);

  for (final T s : array) {
    l.add(s);
  }
  return (l);
}

// for Java < 1.5 (no generics, no compile-time type-safety, boo!)
static List arrayToList(final Object[] array) {
  final List l = new ArrayList(array.length);

  for (int i = 0; i < array.length; i++) {
    l.add(array[i]);
  }
  return (l);
}
```

edited May 16 '13 at 15:54

answered Nov 16 '12 at 17:16

haylem
14.5k 3 41 78

11 +1 But note that the `List` returned by `Arrays.asList` is mutable in that you can still `set` elements - it just isn't resizable. For immutable lists without Guava you might mention `Collections.unmodifiableList` . – Paul Bellora Jan 10 '13 at 2:54

@PaulBellora: Thanks Paul, I had never taken the time to update based on your comment, but this is finally done. You'd have been welcome to do it yourself though, as you were perfectly right. Thanks for fixing typos back then as well. – haylem Apr 17 '13 at 0:14

@haylem In your section *For Educational Purposes: The Good ol' Manual Way*, your `arrayToList` for Java 1.5+ is incorrect. You are instanciating lists of `String` , and trying to retrieve strings from the given array, instead of using the generic parameter type, `T` . Other than that, good answer, and +1 for being the only one including the manual way. – afsantos May 16 '13 at 13:41

@afsantos: ah ah, you're perfectly right indeed. That's what happens when you type stuff on the web without checking it :). Don't know why others rejected your edit, it was correct. Thanks. – haylem May 16 '13 at 15:55

Since this question is pretty old, it suprises me that nobody suggested the simplest form yet:

```
List<Element> arraylist = Arrays.asList(new Element(1),new Element(2),new Element(3));
```

As of Java 5, Arrays.asList() takes a varargs parameter and you don't have to construct the array explicit.

answered Jun 22 '11 at 11:30

Tim Büthe
30.5k 12 50 89

6 Thanks Tim; good point. I have up-voted this but will retain the other accepted answer as the question was specifically about the *conversion* of an array to a List. – Ron Tuffin Jun 22 '11 at 13:47

```
new ArrayList<T>(Arrays.asList(myArray));
```

answered Oct 1 '08 at 14:40

Bill the Lizard
198k 126 431 719

Another way (although essentially equivalent to the `new ArrayList(Arrays.asList(array))` solution performance-wise:

```
Collections.addAll(arraylist, array);
```

edited Apr 3 '12 at 23:28

answered Apr 3 '12 at 23:20

Peter Tseng
5,468 1 23 33

To convert an array to an ArrayList, developers often do this:

```
List<String> list = Arrays.asList(arr);// this is wrong way..
```

`Arrays.asList()` will return an ArrayList which is a `private static class inside Arrays`, it is not the java.util.ArrayList class. `The java.util.Arrays.ArrayList` class has `set(), get(), contains()` methods, but does not have any methods for adding elements, so its size is fixed. To create a real ArrayList, you must do:

```
ArrayList<String> arrayList = new ArrayList<String>(Arrays.asList(arr));
```

The constructor of ArrayList can accept a **Collection type**, which is also a super type for `java.util.Arrays.ArrayList`

edited Nov 20 '14 at 17:16

answered Jun 25 '14 at 7:20

Nepster
7,756 1 58 65

2 It's not the wrong way if you just need a `List`, and don't need an `ArrayList`, which you actually don't most of the time. Most of the time when developers create an `ArrayList` it's out of habit, more than out of need. – Christoffer Hammarström Oct 8 '15 at 11:07

You probably just need a List, not an ArrayList. In that case you can just do:

```
List<Element> arraylist = Arrays.asList(array);
```

answered Oct 1 '08 at 14:45

Kip
47k 66 165 207

6 That will be backed by the original input array, which is why you (probably) want to wrap it in a new ArrayList. – Bill the Lizard Oct 1 '08 at 14:46

11 Be careful with this solution. If you look, Arrays ISN'T returning a true java.util.ArrayList. It's returning an inner class that implements the required methods, but you cannot change the memebers in the list. It's merely a wrapper around an array. – Mikezx6r Oct 1 '08 at 14:47

You can cast the List<Element> item to an ArrayList<Element> – monksy Oct 9 '09 at 22:48

7 @Mikezx6r: little **correction**: it's a fixed-size list. You can change the elements of the list (`set` method), you cannot change the size of the list (not `add` or `remove` elements)! – Carlos Heuberger Dec 4 '09 at 13:10

Yes, with the caveat that it depends on what you want to do with the list. It's worth notng that if the OP simply wants to iterate through the elements, the array doesn't have to be converted at all. – PaulMurrayCbr May 5 '13 at 9:23

Another update, almost ending year 2014, you can do it with Java 8 too:

```
ArrayList<Element> arrayList =
Stream.of(myArray).collect(Collectors.toCollection(ArrayList::new));
```

A few characters would be saved, if this could be just a `List`

```
List<Element> list = Stream.of(myArray).collect(Collectors.toList());
```

answered Nov 25 '14 at 20:48

whyem
635 1 7 20

1 It's probably best not to be implementation-dependent, but `Collectors.toList()` actually returns an `ArrayList`. – bcsb1001 Dec 31 '14 at 19:55

```
// Guava
import com.google.common.collect.ListsLists
...
List<String> list = Lists.newArrayList(aStringArray);
```

answered Jan 7 '14 at 6:04

Bohdan
4,032 4 32 41

If you use :

```
new ArrayList<T>(Arrays.asList(myArray));
```

you **may** create **and fill** two lists ! Filling twice a big list is exactly what you don't want to do because it will create another `Object[]` array each time the capacity needs to be extended.

Fortunately the JDK implementation is fast and `Arrays.asList(a[])` is very well done. It create a kind of ArrayList named Arrays.ArrayList where the Object[] data points directly to the array.

```
// in Arrays
@SafeVarargs
public static <T> List<T> asList(T... a) {
    return new ArrayList<>(a);
}
//still in Arrays, creating a private unseen class
private static class ArrayList<E>

    private final E[] a;
    ArrayList(E[] array) {
        a = array; // you point to the previous array
    }
    ....
}
```

The dangerous side is that **if you change the initial array, you change the List !** Are you sure you want that ? Maybe yes, maybe not.

If not, the most understandable way is to do this :

```
ArrayList<Element> list = new ArrayList<Element>(myArray.length); // you know the initial
capacity
for (Element element : myArray) {
    list.add(element);
}
```

Don't use `Collections` , `Arrays` , or Guava. I love to use it, but if it don't fit, don't use it. Write another inelegant line instead.

Well... still use and learn Guava if you can.

edited Dec 16 '15 at 9:41

rtruszk
3,298 13 22 41

answered Jan 18 '14 at 13:01

Nicolas Zozol
2,345 1 18 39

I fail to see the fundamental difference between your loop at the end of the answer and the `new ArrayList<T>(Arrays.asList(myArray));` part which you discourage to use. Both do quite the same and have the same complexity. – glglgl Feb 22 '15 at 17:03

The Collections one create a pointer at the beginning of the array. My loop create many pointers : one for each array member. So if the original array changes, my poiners are still directed toward the former values. – Nicolas Zozol Feb 22 '15 at 19:59

1 `new ArrayList<T>(Arrays.asList(myArray));` does the same, it copies the `asList` to an `ArrayList` ... – glglgl Feb 22 '15 at 21:56

According with the question the answer using java 1.7 is:

```
ArrayList<Element> arraylist = new ArrayList<Element>(Arrays.<Element>asList(array));
```

However it's better always use the interface:

```
List<Element> arraylist = Arrays.<Element>asList(array);
```

answered May 17 '15 at 22:07

nekperu15739
194 1 4

This answer adds nothing that has not already been said in other answers (that have been here for 6-7 years) – Tim Castelijns Dec 16 '15 at 9:50

If your `Array` is of type `String` ,

```
String str[] = {"a","b","c","d","e"};
List<String> list = new ArrayList(Arrays.asList(str));
```

To learn more about `ArrayList` , refer this link

edited Dec 16 '15 at 9:42

rtruszk
3,298 13 22 41

answered Nov 17 '15 at 6:54

satish
159 1 3

1 This answer adds nothing that has not already been said in other answers (that have been here for 6-7 years), other than that, the quotes are wrong – Tim Castelijns Dec 16 '15 at 9:50

Another simple way is to add all elements from the array to a new ArrayList using a for-each loop.

```
ArrayList<Element> list = new ArrayList<>();

for(Element e : array)
    list.add(e);
```

answered Dec 16 '15 at 20:32

spencerlarry
484 2 12

Pretty simple:

Make your array of elements:

```
Element[] elements = {new Element(1), new Element(2), ...};
```

and make an `ArrayList` with the `Arrays.asList` function.

```
ArrayList<Element> list = Arrays.asList(elements);
```

Hope this helps!

answered 18 hours ago

codecubed
21 2

Simplest way todo it `asList` method of `Arrays` class.

```
Arrays.asList(array);
```

answered Oct 28 '15 at 11:24

OAD
876 4 23

It doesn't produce an ArrayList but an unmutable implementation instead. See this answer: stackoverflow.com/a/24402502/1892137 – Leandro Glossman Dec 7 '15 at 19:57

There is another option if your goal is to generate a fixed list at runtime, which is as simple as it is effective:

```
static final ArrayList<Element> myList = generateMyList();

private static ArrayList<Element> generateMyList() {
  final ArrayList<Element> result = new ArrayList<>();
  result.add(new Element(1));
  result.add(new Element(2));
  result.add(new Element(3));
  result.add(new Element(4));
  return result;
}
```

The benefit of using this pattern is, that the list is for once generated very intuitively and therefore is very easy to modify even with large lists or complex initialization, while on the other hand always contains the same Elements on every actual run of the program (unless you change it at a later point of course).

edited Oct 5 '13 at 10:17

answered Oct 4 '13 at 21:50

TwoThe
6,067 1 9 31

2 This doesn't answer the original question. The OP already has the elements in a container, which we can assume has random contents. This approach depends on the list needing the exact same elements every time this code is run. Also, the overuse of the static keyword is bad practice, and there are many other ways of doing this in practice which involve less boilerplate code. – Shotgun Ninja Apr 8 '15 at 15:17

---

protected by Robert Harvey ♦ May 11 '11 at 3:26

Thank you for your interest in this question. Because it has attracted low-quality or spam answers that had to be removed, posting an answer now requires 10 reputation on this site.

Would you like to answer one of these unanswered questions instead?